

Final Project Report: Budget Buddy

Course: SWE 632 — User Interface Design and Development **Team:** Laws Smith, Kevin Le, Samana Hussain **Semester:** Spring 2026

1. Project Name & Value Proposition

Project name: Budget Buddy

Value proposition: A mobile budgeting app that helps people stay on track with monthly category budgets by keeping track of how much spending they have left, their transaction history, and real-time alerts to keep budgeting easy.

2. Team Members and Roles

All three of us contributed to every phase of the project. There was no single lead on any phase.

3. Problem / Solution Overview

People want to be better at budgeting either to cut costs or save for the future, but existing apps are scattered across disconnected tools, demand too much manual categorization, and mostly only update users after the money has already been spent. Budget Buddy targets the window in between, where a budget can still help shape both long-term savings and day-to-day spending. It tracks per-category monthly budgets, translates remaining dollars into a daily spending pace, and alerts users when a budget reaches a certain threshold so users aren't caught off guard.

4. Needfinding

Budget Buddy started from a personal need that Laws articulated in his Module 3 individual assignment. He had tried Quicken, Mint, YNAB, Rocket Money, and a rotation of bank apps over the years and found that none of them were both real-time and all-inclusive in the way he wanted. The problem was existing tools force users to maintain too many disconnected spaces, require too much manual categorization, and only update users after money has been spent. Kevin and Samana joined the concept as a group project from there.

4a. Interviews

We did not run any interviews in the needfinding phase of this project. However, Laws had identified a gap in the budget tracking market prior to this project through informal conversations with friends and family. While this is not the best approach to needfinding, it fits within the restraints of this project.

4b. Synthesis

Since we were not able to run formal needfinding interviews for this project, we instead synthesized insights from prior informal conversations as well as personal experiences with budgeting apps. Through this process, we found users want a convenient and low effort budgeting app that automates all of the menial parts of budgeting.

5. Design Evolution

5a. Final Solution

Budget Buddy is a mobile budget tracker organized around five sections: a home page summarizing the month, a budgets list, a transactions feed, an alerts feed, and settings. Users are able to connect their bank accounts to the app so transactions are automatically tracked in the app, without needing to manually input anything. Users can create budgets for categories, like “food” or “clothing”. Each budget consists of an amount as well as an alert threshold, which is a percentage of the budget amount. The user is alerted when the alert threshold is reached, ensuring users are well informed about their budget.

Key flows:

- See the month at a glance from the homepage.
- Select a category to see related transactions and adjust the budget.
- Add a transaction quickly via the Quick Actions card.
- Get alerted when a category is trending over budget early enough to act.

The final design is the result of multiple iteration cycles reflecting the design principles covered in class. Some of the principles that had an especially large impact on our design include:

1. Error prevention: We used this principle when adding a confirmation page to our budget creation flow, allowing users to catch any typos in their budget before it's too late. We also make sure every value entered in our app is valid. For example, a budget can not have a negative budget amount. This again makes sure budgets are correct so users can easily watch their spending.
2. Show an overview of information, and let users filter for specific tasks: We used this principle when redesigning our transactions page. Originally, each transaction list item consisted of just a category, date, and amount. We also only kept track of transactions for the month. During our iterating we realized users may want a wider overview of their transactions than just the current month, leading us to save previous months' transactions as well. We also realized users may want to filter through the transactions on a more granular level, so we added a search function and merchant names for each transaction.

5b. Tasks

In Module 6, we did a usability study around four core tasks that also shaped the final prototype. They are as follows:

Task 1: Create new budgets in areas you tend to overspend.

Set up a monthly clothing budget and a monthly food budget in the app. For this task, you will need to set \$300 for clothing and \$400 for food.

- Why it matters: Users want to regularly create and adjust budgets as their goals change. The flow needs to be simple enough to be easily understood as well as careful enough to prevent mistakes.
- Annotated task flow: (*Budgets list* → *"Create a Budget"* → *New Budget form* → *review/confirm step* → *Budget Created success* → *back to Budgets list*)

1:03

Create New Budget

CATEGORY

Clothing

You already have a Clothing budget (\$300/mo)
Creating another will replace your existing Clothing budget and its tracked spending.

MONTHLY BUDGET AMOUNT

300

Enter the amount you want to spend per month in this category.

START DATE

February 1, 2026 Tap to change

Budget periods start on the 1st by default. You can change this if needed.

ALERT ME WHEN I'VE SPENT

50% \$150.00 75% \$225.00 **SELECTED** 90% \$270.00

You'll get a notification when you've spent \$225.00 of your \$300.00 budget, with \$75.00 remaining.

Review Budget

Image: The “Create New Budget” form

Task 2: Check your clothing budget

It's a few weeks later and you've been spending normally. You're out shopping later this week and see a shirt you like for \$36.00. Before buying it, use the app to figure out whether your budget has enough room for the purchase.

- Why it matters to the user base: Users want to be able to quickly and efficiently see the current state of their budget in order to make purchasing decisions. Our app shows the remaining balance of the budget to help users know how much they can spend real-time.
- Annotated task flow: (*Home page* → *Clothing Budget detail*)

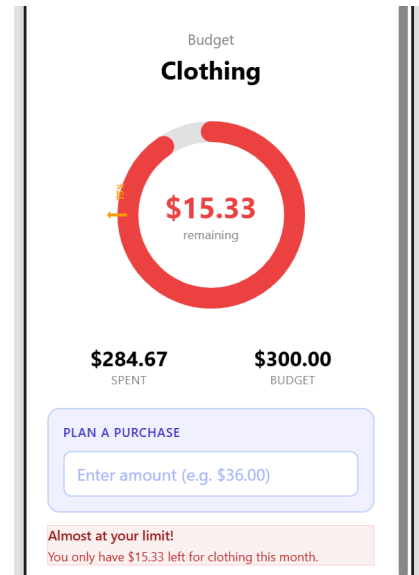


Image: The Clothing Budget page

Task 3: Check your food budget

After shopping, you realize you are hungry and want to grab lunch for about \$20. Check your food budget in the app to see if that's a reasonable spend or if you need to find a cheaper option.

- Why it matters to the user base: Same as the clothing, users want to see how much they can spend real-time
- Annotated task flow: (*Home page* → *Food Budget detail*)

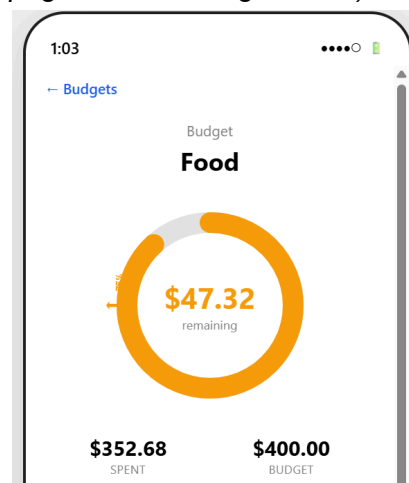


Image: The Food Budget page

Task 4: Review transactions

After buying lunch, you want to make sure the app picked up the purchase. Review your recent transactions, check and see if your recent purchases have been updated correctly in your budgeting app.

- Why it matters to the user base: Users want to make sure they are operating at the latest budget, so checking to see if recent transactions have processed gives them that security.
- Annotated task flow: (Notification or Home page → Food Budget detail → Recent Transactions or Home page → Transactions)

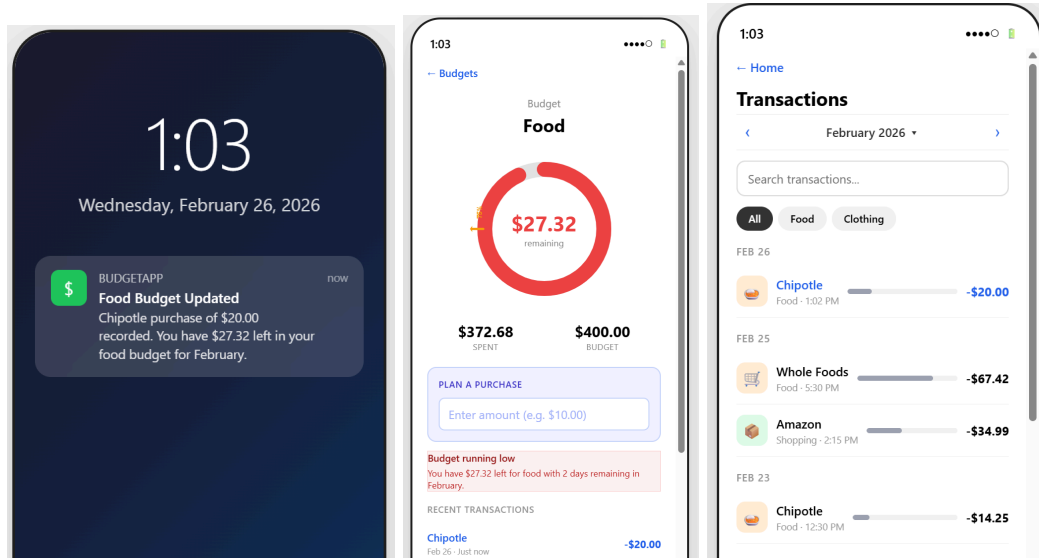


Image: Recent Transactions. The left images shows the budget notification, the middle image shows the recent transaction in the food budget dashboard and the right images shows the transactions dashboard.

5c. Design Evolution Visualizations

For each iteration we describe the evaluation technique used, what was learned, and how the design changed in response. The full set of weaknesses identified in each iteration lives in that iteration's deliverable PDF. This report focuses on a few representative changes per iteration to show the evolution.

Iteration 1: Module 5 Wireframes (initial prototype)

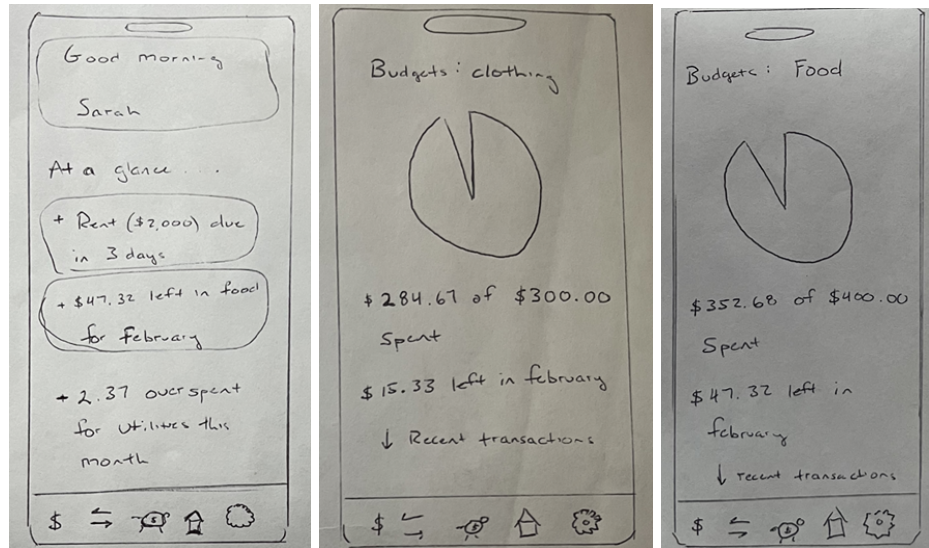


Image: Hand-drawn wireframes of the original Budget Buddy concept. The at-a-glance Dashboard greets Sarah and lists upcoming bills (Rent \$2,000 due in 3 days), budget status (\$47.32 left in food for February), and over-spending alerts (\$2.37 overspent on utilities); the Clothing Budget donut shows \$15.33 of \$300 left for the month; the Food Budget donut shows \$47.32 of \$400 left.

Budget Buddy began here as Laws's individual M5 assignment. The hand-drawn wireframes laid out the original concept: a personal-finance app organized around per-category monthly budgets, anchored by the Sarah scenario showing the in-the-moment-of-decision use case the product is built around. The wireframes covered the Dashboard, individual budget detail screens (Clothing, Food) with donut charts and remaining balances, a push-notification flow after a purchase, and the post-purchase updated budget view. The evaluation here was scenario-driven design rather than a structured critique. The main thing learned was that grounding the design in a single concrete scenario (Sarah, end of month, deciding whether to buy a shirt) concretized the value proposition and gave the team a single user story to design and evaluate against from M6 onward. The Sarah scenario, donut charts, push notifications, and per-category budget structure all originated here and survived to the final design.

Iteration 2: Module 6 Usability Study

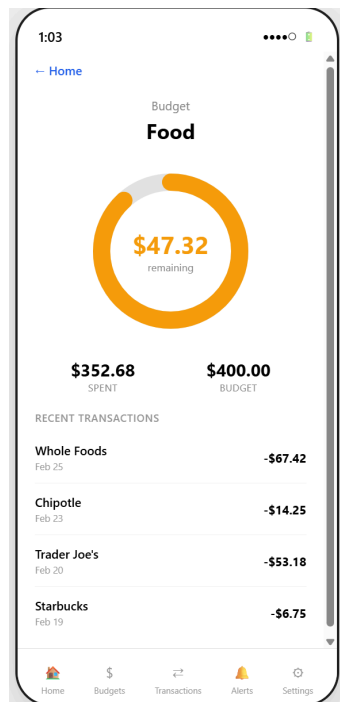


Image: Interactive HTML wireframe of the Food Budget screen as shown to participants — donut chart, "\$47.32 remaining" callout, spent vs. budget totals (\$352.68 of \$400), and a recent-transactions list.

The first group iteration on Budget Buddy. We converted Laws's M5 hand-drawn wireframes into interactive HTML wireframes with facilitator-operated Wizard of Oz navigation between 11 screens, then ran a four-participant think-aloud usability study (participants A, B, C, D) covering three tasks: check budget before a purchase, create a new budget with an alert, and re-categorize a transaction. Each member ran at least one session, took critical-incident notes, and contributed to the synthesis of five usability issues. The most consequential findings were that users had no way to *preview* how a purchase would affect a budget (they could only see the impact after the fact), transaction labels were too vague to verify against real-world activity ("Lunch" instead of "Chipotle"), and the budget-creation flow had several confusing moments around alert thresholds and button styling. These findings fed directly into the M8 interaction iteration.

Iteration 3: Module 8 Interaction Iteration

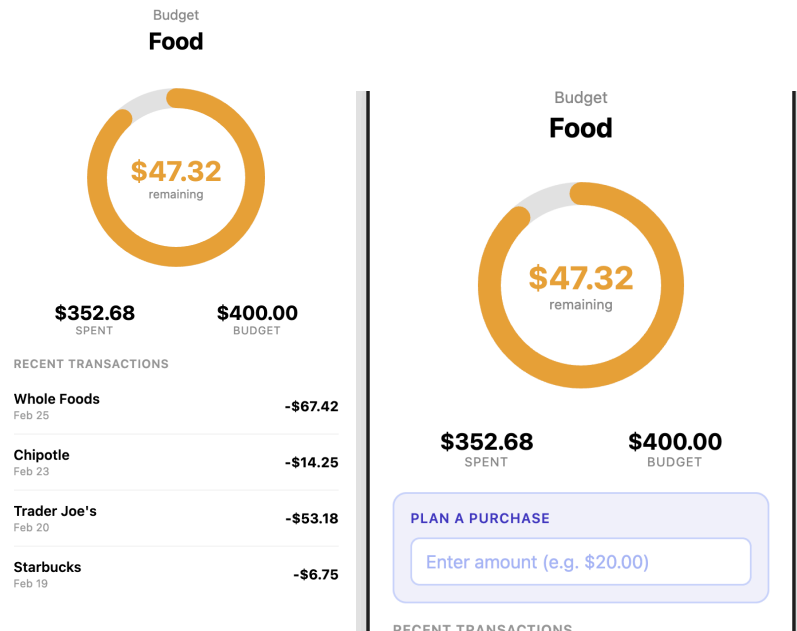


Image: Before/after pair of the budget detail screen. Before (left): only "\$47.32 remaining" with no way to test a hypothetical purchase. After (right): a "Plan a Purchase" card lets the user enter a hypothetical amount (e.g. \$36.00) and immediately see the projected remaining balance with an "Almost at your limit!" warning when the purchase would push the category close to its threshold.

In module 8, we updated our app with usability issues we identified in Module 6 Group Assignment 1. The first issue we fixed was the inability to see how a purchase would affect a budget before actually making a purchase. We fixed this by letting users manually input potential purchases to see how their budget is affected. We then found transaction labels are too vague, making it difficult to verify purchases. We fixed this by replacing vague labels, like "lunch", with more merchant specific labels, like "Chipotle". And finally, we found the budget creation work flow was not clear. We fixed this by adding more context to the options presented in the process. After implementing these fixes, the design of our app became much less frustrating to use.

Iteration 4: Module 9 Interaction Design Critique & Iteration

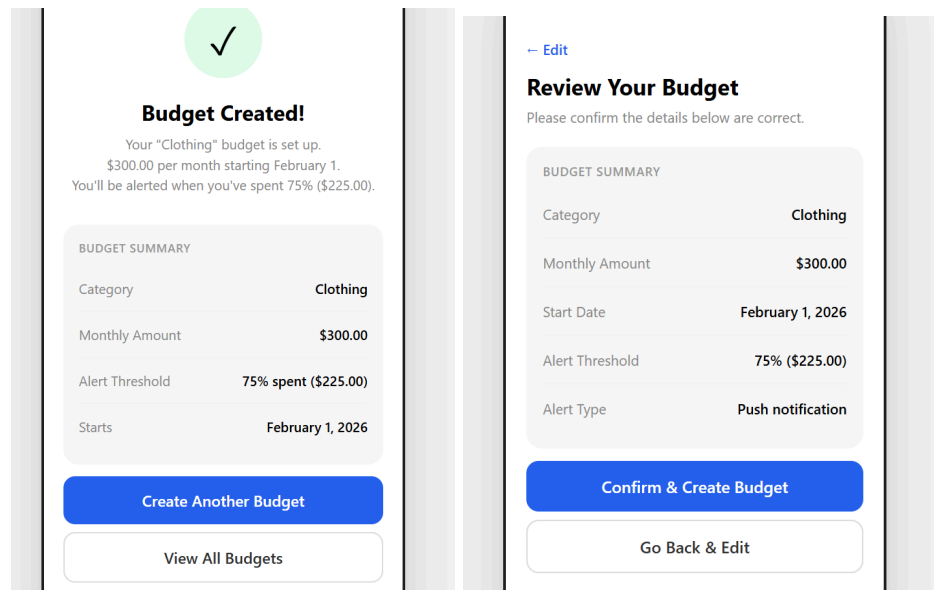


Image: Before/after pair of the budget creation flow. Before (left): tapping "Create Budget" commits the budget instantly with no review. After (right): a confirmation step appears summarizing the chosen category, monthly amount, start date, and alert threshold, with explicit "Confirm" and "Go Back" buttons before the budget is finalized.

In module 9, we evaluated our app based on site design, interaction techniques, and error prevention. With these ideas in mind, we found 7 issues in our app. One of the more immediate issues discovered was that there was no confirmation page when creating a budget. This means users have no way to correct any errors that may have occurred during budget creation. To fix this, we created a confirmation page that shows all the selected values for a budget, allowing users to review their inputs and fix values if necessary. Another issue we encountered was that our navigation bar did not indicate where users were in our app. We fixed this by highlighting the active navigation item while leaving the inactive items uncolored. From this iteration cycle we learned that creating an app that follows the guidelines outlined in the site design, interaction techniques, and error prevention modules significantly reduces frustrations felt by end users. At the end of this iteration cycle, our app became more forgiving to mistakes and gave users a better understanding of their current position within the app.

Iteration 5: Module 10 Interaction Design Critique & Iteration

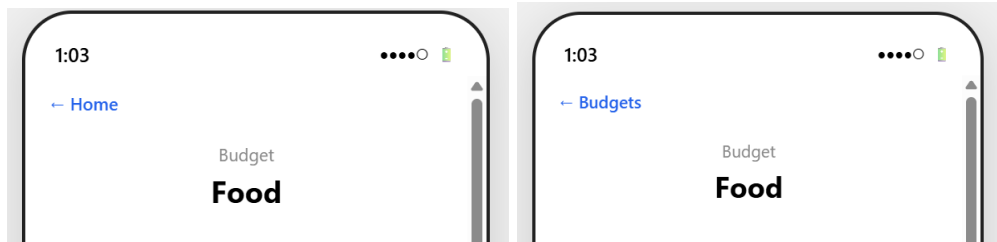


Image: Before/after pair of a budget detail screen's back button. Before (left): "← Home" returns the user to the Dashboard, breaking the expected hierarchy. After (right): "← Budgets" returns to the Budgets list, matching where the user came from and matching standard mobile-nav conventions.

START DATE

February 1, 2026

START DATE



February 1, 2026

Tap to change

Budget periods start on the 1st by default. You can change this if needed.

Image: Before/after pair of Create New Budget screen Start Date section. pre fills in the start date of any new budgets you make with the first day of the month. However, there is no indicator that users can edit this date. After, The start date box now has a label indicating that the start date can be changed by tapping on the box. There is also a calendar icon that generally signifies a date is editable. These changes clearly convey to users the start date is editable.

In Module 10, a key lesson was that usability issues often stem less from missing features and more from mismatches between user expectations and system feedback. Several of the original weaknesses revealed gaps in affordance, consistency, and recognition. For example, non-functional navigation tabs created uncertainty about whether actions were registered, while duplicated dashboard shortcuts and misleading back navigation disrupted users' mental model of the app's structure. In response, the design was updated to ensure that every interactive element produces immediate, meaningful feedback and follows a consistent navigation hierarchy, aligning with principles from Norman and Krug about visibility and predictable system behavior. Another important takeaway was the importance of reducing cognitive load through clearer signifiers and recognition-based design. Issues such as ambiguous date editing, color-only alert selection, and text-heavy

transaction lists all required users to rely on memory or guesswork rather than clear visual cues. The redesign addressed this by adding explicit indicators (such as editable date icons, checkmarks for selection states, and category icons for transactions), making interactions more discoverable and accessible.

Iteration 6: Module 13 Visual Design Critique & Iteration

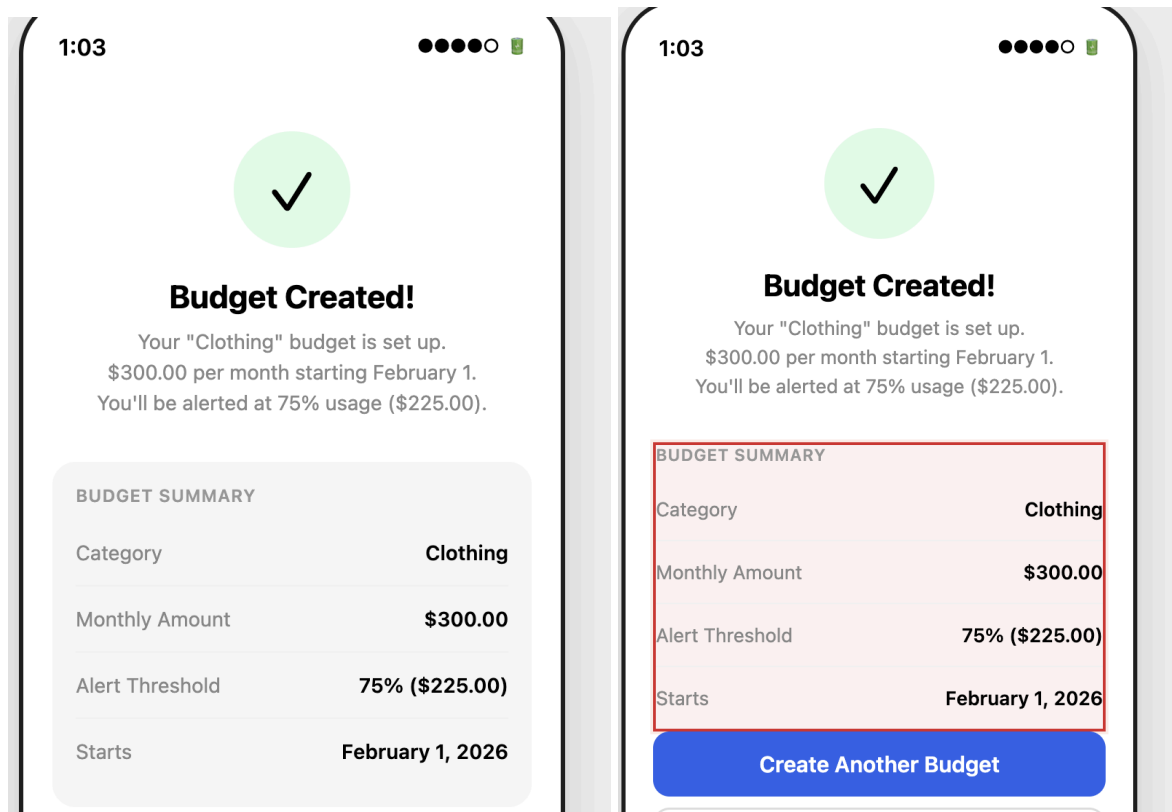


Image: Before/after pair of the Dashboard alerts area. Before (left): critical alerts are sized and weighted the same as routine notifications, so a budget about to overflow blends visually with low-priority status updates. After (right): critical alerts are scaled up, given heavier visual weight, and elevated in the layout so they read as more important at a glance, applying Mullet & Sano's principle that visual hierarchy should match informational hierarchy.

1:03



**Good Morning,
Sarah**

Here's your financial snapshot

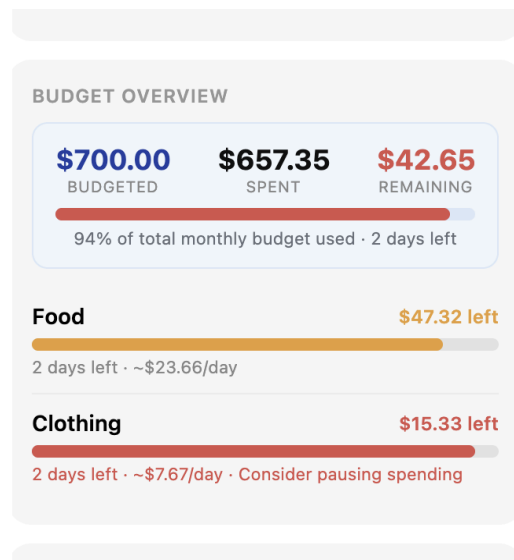
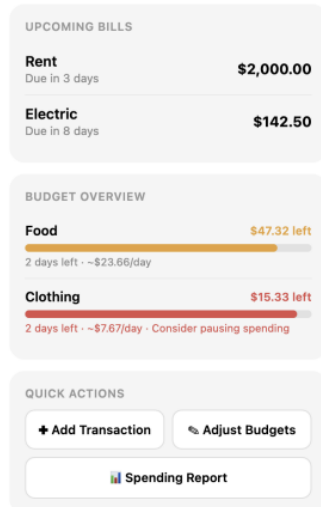


Image: Before/after overview of budget page. In the information visualization reading we learn that information visualizations should provide an overview of the information to users then allow users to filter the information for specific tasks. In the previous form our app did not provide an overview of the user's budget. It displayed each individual budget, but not total spending and total money remaining to be spent. The new budget overview section now displays total budget, spend, and remaining budget.

In Module 13, Across these design weaknesses, we learned that effective financial interfaces must prioritize clear information hierarchy, strong visual encodings, and meaningful system feedback rather than simply displaying raw data. Several issues stemmed from failing to support the full “overview → detail → filter” workflow emphasized in information visualization principles. For example, the absence of a budget aggregate overview and limited transaction filtering showed that users need both high-level context and the ability to drill down into specific data. We also learned that visual variables such as size, position, color, and depth must be used intentionally, otherwise users are forced to do unnecessary cognitive work, as seen in the original transactions list and flat, undifferentiated card layouts. In response, the design was revised to better align with usability and visualization principles by making key information more discoverable, interpretable, and accessible. A new budget overview consolidates total spending and remaining balance, while transaction filtering and month navigation support flexible exploration of financial history. Visual encodings were strengthened by introducing proportional bar lengths for transaction amounts, explicit alert severity labels, and clearer alert hierarchy through contrast and emphasis. Overall, the redesign shifts the system from a flat

data display into a structured, readable, and actionable budgeting tool that better supports real user decision-making.

6. Final Prototype Implementation

6a. Tools Used

Tool	Purpose	Pros	Cons
HTML / CSS / JavaScript	Creating interactive prototype of our app	fast iteration, demonstrable in any browser, easy version control	Not a real mobile app; interactions are simulated, no persistence between sessions
AI-assisted code generation (Claude)	Translating design changes from sketch and spec into working prototype	Fast turnaround on visual revisions, freed time for design thinking	Required careful review; generated code occasionally introduced unintended visual changes outside the requested fix
Git + GitHub	Source control for iterations and team coordination	Accessible history of every change, ability to roll back changes	Not everyone is familiar with Github, some start up difficulties

6b. Wizard of Oz Techniques

The mockup simulates several behaviors that would require real backend infrastructure:

- Transaction data is hard-coded in the HTML. Adding a new transaction in the prototype does not persist between sessions.
- Alerts are pre-populated from a fixed dataset rather than generated from real spending behavior.
- The daily-allowance calculation uses a static "today's date" baked into the page rather than the real current date.

6c. Hard-Coded Techniques

- All budget categories, amounts, and thresholds are static in the HTML.
- Navigation between screens is implemented by toggling CSS visibility, not real routing.
- The before and after mockups are separate files rather than a single app with a versioned UI.

7. Reflection & Next Steps

7a. Main Learnings

From the design thinking process:

- The biggest gains came from cycles where we evaluated against an outside framework (Nielsen heuristics, Krug, Norman, Mullet & Sano) rather than relying on team intuition. Every iteration that started with "what principle is this violating?" produced clearer fixes than ones that started with "what should we change?"
- Iteration is not free. By M10 we were re-discovering issues we had already caught, a sign that we needed to consolidate fixes between rounds rather than treat each module's critique as fresh ground.

From our specific project:

- Budget tracking sits in a crowded space (Mint, YNAB, Rocket Money, every bank app). Our differentiator emerged through iteration: spending pace, not just spending balance. That was not obvious in the M5 wireframes or even after the M6 usability study.
- Visual design was where the biggest perceived-quality gains happened. M9 and M10 fixed real interaction bugs, but M13's visual changes are what made the app feel like something we would want to use.
- User interface design can be just as important as purpose. Even if an app solves a problem, users may not use the app if it is frustrating to use.

What stood out the most across this project was how much small, detail-level design decisions, like button states, contrast, and navigation consistency, can significantly impact the overall usability of an app. Evaluating designs through established principles rather than intuition consistently led to clearer, more effective improvements.

7b. If We Had More Time

- Predictive alerts: Instead of alerting after a threshold is crossed, push "at this pace you'll be over by end of month" warnings earlier.
- Customizable categories and rollover. Users with non-standard spending patterns need flexibility we didn't get to.
- Accessibility audit: We addressed color-blindness in the alert cards in M13, but a full study with users who have accessibility issues would potentially uncover more issues.
- Primary user research: Specific studies that would shape the next version could look into how many push notifications are helpful before they cause too much stress, what makes a budgeting app stay in someone's daily routine versus get abandoned after a few weeks, the mental models people use for budgeting and saving money, which common

expenses are easiest to cut to get initial buy-in for a new budget, and pricing research on whether users would pay a subscription, accept ads, or expect a freemium model.